

Pruebas de Software: Pruebas Mutación y Mock

Semana 11

Ingeniería de Software 2

2026-1



Logro de la sesión



Los estudiantes comprenderán el uso de mocks para simular comportamientos en pruebas unitarias, facilitando el aislamiento y la verificación de interacciones. Aprenderán sobre las pruebas de mutación, que evalúan la efectividad de las pruebas automatizadas al introducir cambios en el código. Además, explorarán los diferentes tipos de pruebas de integración, lo que les proporcionará una base sólida para aplicar estas técnicas en el desarrollo de software.

- ¿Cómo puedes simular el comportamiento de componentes sin depender de su implementación real?
- ¿Sabías que las pruebas de mutación pueden ayudarte a identificar debilidades en tus pruebas automatizadas?
- ¿qué tipo de pruebas de integración son más efectivas para garantizar que diferentes partes de tu aplicación funcionen juntas sin problema



Contenidos

- Framework para mocks.
- Fundamentos de pruebas de mutación.
- Tipos de pruebas de integración.

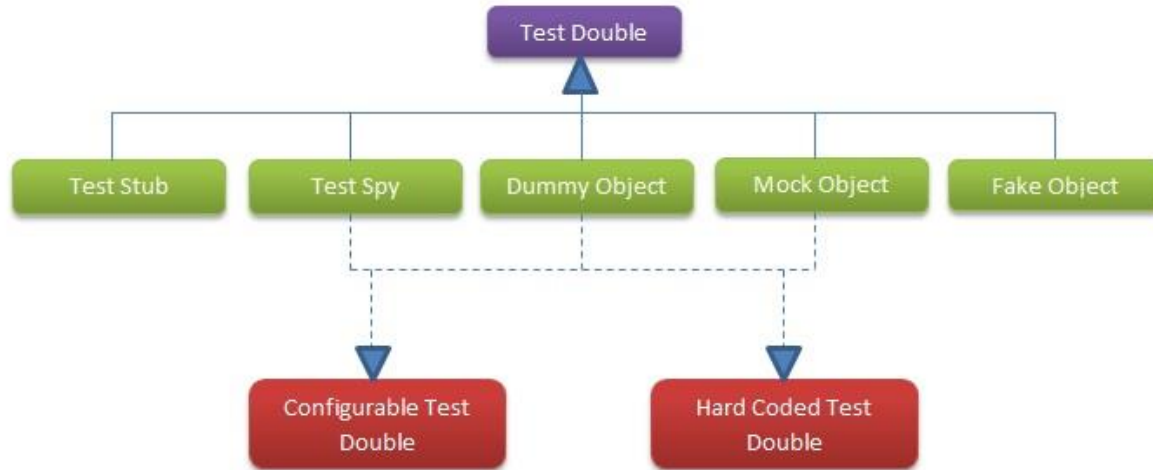
Framework para mocks

Los mocks son herramientas esenciales en el desarrollo de software, especialmente en pruebas unitarias, ya que permiten simular el comportamiento de componentes o servicios externos.



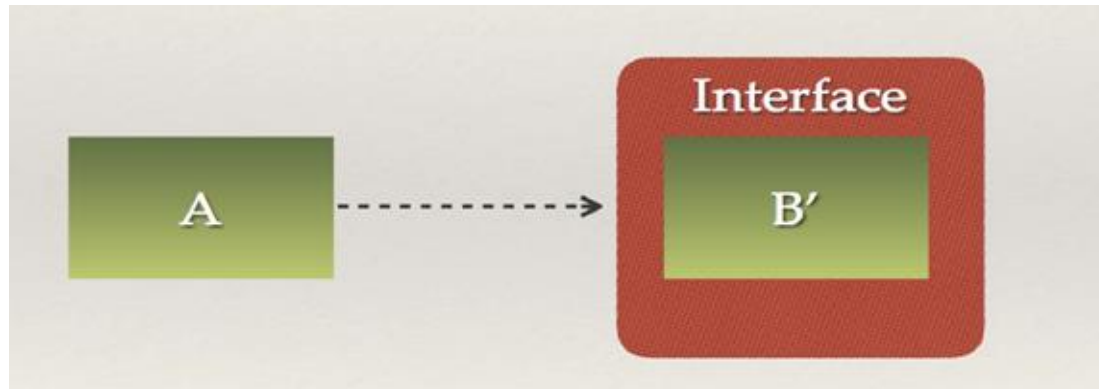
Dobles de Test

Los dobles de test son objetos que simulan el comportamiento de componentes reales en pruebas de software. Se utilizan para aislar la unidad de código que se está probando, permitiendo verificar su comportamiento sin depender de las implementaciones reales de sus dependencias.



Mocks

Los mocks son objetos que registran las interacciones que tienen con el código bajo prueba. Permiten verificar si se llamaron ciertos métodos y con qué argumentos. Son útiles para comprobar que el código interactúa correctamente con sus dependencias.



Mocks - Beneficios

- **Aislamiento de Pruebas:** Permiten centrar las pruebas en la lógica del código, eliminando dependencias externas.
- **Simulación de Comportamientos:** Facilitan la creación de escenarios específicos sin implementar la lógica completa de las dependencias.
- **Verificación de Interacciones:** Ayudan a comprobar que se realizan las interacciones esperadas con las dependencias.
- **Reducción de Costos de Prueba:** Evitan la necesidad de entornos de prueba complejos, acelerando el proceso de pruebas.
- **Mejora de la Cobertura de Pruebas:** Hacen más accesibles los casos difíciles de alcanzar, aumentando la cobertura de pruebas.
- **Facilita el Desarrollo Ágil:** Permiten realizar pruebas rápidas y efectivas en entornos de desarrollo en constante cambio.



Proceso para usar Mock Objects

1. Crear los *Mock Objects* necesarios
2. Definir el comportamiento esperado de los *Mock Objects*, estableciendo las expectativas
3. Crear la instancia de la clase que va a ser probada de manera que use las instancias de los *Mock Objects* en vez de los objetos colaboradores originales
4. Ejecutar el método que se vaya a probar realizando la prueba correspondiente
5. Decir a cada *Mock Object* involucrado en la prueba que verifique si se cumplieron las expectativas



EASYMOCK



Example

```
public class InternetRelayChatTest extends TestCase {  
    public void testJoin() {  
        String msg = "Hello";  
        Client fred = EasyMock.createMock(Client.class);  
        Client peter = EasyMock.createMock(Client.class);  
        Client mary = EasyMock.createMock(Client.class);  
  
        expect (mary.onMessage("homer",msg)).andReturn(true);  
        peter.onMessage("homer",msg);  
        expectLastCall().andReturn(true);  
  
        replay(peter, fred, mary);  
  
        InternetRelayChat irc = new InternetRelayChat();  
        irc.join("the boss", peter);  
        irc.join("cool", mary);  
        Prompt prompt = irc.join("homer", fred);  
        prompt.say(msg);  
  
        verify(peter, fred, mary);  
    }  
}
```

1

2

3

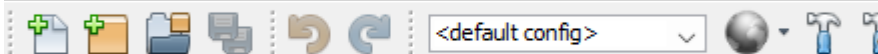
4

5

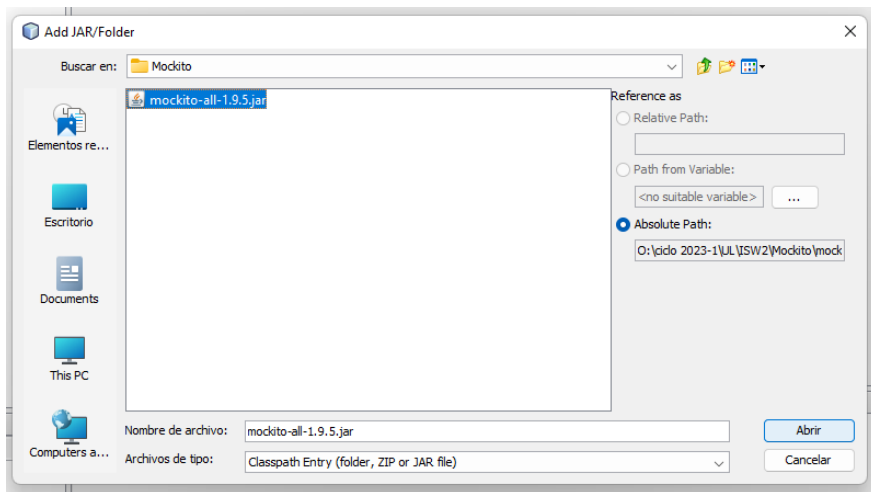
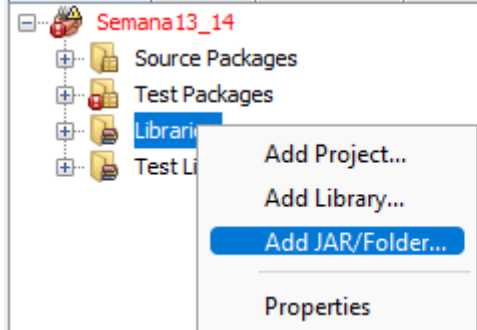
Importando Mockito

Semana13_14 - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools



Projects Files Services



Ejemplo Mockito

```
1 package semana13_14;
2
3 import javax.security.cert.X509Certificate;
4 import java.security.Principal;
5 import java.sql.Connection;
6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9
10 public class LoginController {
11     private Connection connection;
12
13     public LoginController(Connection connection) {
14         this.connection = connection;
15     }
16
17     public boolean login(X509Certificate certificado) {
18         boolean resultado = false;
19         PreparedStatement prepareStatement = null;
20         ResultSet resultSet = null;
21         try {
22             Principal subjectDN = certificado.getSubjectDN();
23             prepareStatement = connection.prepareStatement("SELECT count(*) FROM usuarios where login = ?");
24             prepareStatement.setString(1, subjectDN.getName());
25             resultSet = prepareStatement.executeQuery();
26             if (resultSet.next()) {
27                 resultado = resultSet.getInt(1) > 0;
28             }
29         } catch (Exception e) {
30             e.printStackTrace();
31         } finally {
32             try {
33                 if (prepareStatement != null) {
34                     prepareStatement.close();
35                 }
36                 if (resultSet != null) {
37                     resultSet.close();
38                 }
39             } catch (SQLException e) {
40                 e.printStackTrace();
41             }
42         }
43     }
44     return resultado;
45 }
46 }
```

Ejemplo Mockito

```
1 package semanal3_l4;
2
3 import static org.junit.Assert.*;
4 import static org.mockito.Mockito.*;
5 import java.security.Principal;
6 import java.security.cert.CertificateException;
7 import java.sql.Connection;
8 import java.sql.PreparedStatement;
9 import java.sql.ResultSet;
10 import java.sql.SQLException;
11 import javax.security.cert.X509Certificate;
12 import org.junit.Test;
13
14 public class conMockitoTest {
15     @Test
16     public void testLoginConCertificado() throws CertificateException, SQLException
17     {
18         //prepar los mocks
19         X509Certificate certificado=mock(X509Certificate.class);
20         Connection conectar = mock(Connection.class);
21         Principal principal=mock(Principal.class);
22         PreparedStatement preparedStatement=mock(PreparedStatement.class);
23         ResultSet resultSet=mock(ResultSet.class);
24
25         // las expectativas/ programar respuestas
26         when(certificado.getSubjectDN()).thenReturn(principal);
27         when(principal.getName()).thenReturn("Usuario1");
28         when(conectar.prepareStatement(anyString())).thenReturn(preparedStatement);
29         when(preparedStatement.executeQuery()).thenReturn(resultSet);
30         when(resultSet.next()).thenReturn(true);
31         when(resultSet.getInt(1)).thenReturn(1);
32
33         // Probar
34         LoginController controller=new LoginController(conectar);
35         boolean login=controller.login(certificado);
36
37         assertTrue(login);
38     }
39 }
40 }
```



```

1 package semanal3_14;
2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.SQLException;
6 import java.util.Date;
7
8 public class TareaRepository {
9
10     private Connection connection;
11
12     public TareaRepository(Connection connection) {
13         this.connection = connection;
14     }
15
16
17     public void save(Tarea tarea) {
18         PreparedStatement prepareStatement = null;
19         try {
20             Date ahora = new Date();
21             tarea.setFechaCreacion(ahora);
22             prepareStatement = connection.prepareStatement("INSERT INTO tareas (nombre, fecha) VAL
23
24             prepareStatement.setTimestamp(2, new java.sql.Timestamp(ahora.getTime()));
25             prepareStatement.executeUpdate(); // llama al metodo que actualiza
26         } catch (Exception e) {
27             e.printStackTrace();
28         } finally {
29             try {
30                 if (prepareStatement != null) {
31                     prepareStatement.close();
32                 }
33                 if (connection != null) {
34                     connection.close();
35                 }
36             } catch (SQLException e) {
37                 e.printStackTrace();
38             }
39         }
40     }
41 }
42
43 }
```

```

1 package semanal3_14;
2
3 import java.util.Date;
4
5 public class Tarea {
6
7     private String nombre;
8     private Date fechaCreacion;
9
10     public Date getFechaCreacion() {
11         return fechaCreacion;
12     }
13
14     public String getNombre() {
15         return nombre;
16     }
17
18     public void setNombre(String nombre) {
19         this.nombre = nombre;
20     }
21
22     public Tarea(String nombre) {
23         this.nombre = nombre;
24     }
25
26     public void setFechaCreacion(Date fechaCreacion) {
27         this.fechaCreacion = fechaCreacion;
28     }
29
30 }
31 }
```

Verificación de Mocks

```
1 package semanal3_14;
2
3 import static org.mockito.Mockito.*;
4 import java.sql.Connection;
5 import java.sql.PreparedStatement;
6 import java.sql.SQLException;
7 import org.junit.Test;
8 import org.junit.Assert;
9
10
11 public class VerificacionDeMocksTest {
12
13     @Test
14     public void testVerificarLaLlamadaAlMockDePreparedStatement() throws SQLException {
15         // Preparar
16         Tarea tarea = new Tarea("Aprender Mock.verify()"); //esta clase
17         Connection connection = mock(Connection.class); // se crea objeto emulado
18         PreparedStatement preparedStatement = mock(PreparedStatement.class);
19         when(connection.prepareStatement(anyString())).thenReturn(preparedStatement);
20
21         // Probar
22         TareaRepository repository = new TareaRepository(connection);
23         repository.save(tarea); // pasamos el objeto emulado para probar el metodo save
24
25         // Dado que save() no devuelve nada,
26         // uso mock.verify() para saber si se ha guardado la Tarea
27         verify(preparedStatement).executeUpdate(); // estamos verificadon que el preparedStatement a llmado al executeUpdate
28         // verify(preparedStatement).setString(1, "Aprender Mock.verify()");//es adicional***
29     }
30
31 }
32
```



Excepciones Mockito

```
1 package semanal3_l4;
2
3 import static org.mockito.Mockito.*;
4 import java.sql.Connection;
5 import java.sql.PreparedStatement;
6 import java.sql.SQLException;
7 import org.junit.Test;
8
9 public class LanzarExepcionPorMock {
10
11     @Test
12     public void testVerificarQueSeCierraLaConexionEnCasoDeError() throws SQLException {
13         // Preparar
14         Tarea tarea = new Tarea("Aprender Mockito.verify()");
15         Connection connection = mock(Connection.class);
16         PreparedStatement preparedStatement = mock(PreparedStatement.class);
17         when(preparedStatement.executeUpdate()).thenThrow(new SQLException("Excepcion lanzada por Mockito de forma intencionada"));
18         when(connection.prepareStatement(anyString())).thenReturn(preparedStatement);
19
20         TareaRepository repository = new TareaRepository(connection);
21         repository.save(tarea);
22
23         // verificar llamadas a métodos que no se programo
24         verify(connection).close();
25         verify(preparedStatement).close();
26
27         // Ejercicio: Nota que estos asserts fallarian, ¿Por qué?
28         //assertTrue(connection.isClosed());
29         //assertTrue(preparedStatement.isClosed());
30     }
31 }
```



Crear Espias

```
2
3 import static org.mockito.Mockito.*;
4 import java.sql.Connection;
5 import java.sql.PreparedStatement;
6 import java.sql.SQLException;
7 import org.junit.Test;
8 import org.mockito.Mockito;
9
10 public class crearEspia {
11
12     @Test
13     public void testEspiarElMetodoGetNombreDeLaTarea() throws SQLException {
14         // Preparar
15         Tarea tarea = new Tarea("Aprender Mock.verify()");
16         Tarea spy = Mockito.spy(tarea);
17
18         Connection connection = mock(Connection.class);
19         PreparedStatement preparedStatement = mock(PreparedStatement.class);
20         // when(preparedStatement.executeUpdate())
21         // .thenThrow(new SQLException("Excepcion lanzada por Mockito de forma intencionada"));
22         when(connection.prepareStatement(anyString())).thenReturn(preparedStatement);
23
24         // Probar
25         TareaRepository repository = new TareaRepository(connection);
26
27         repository.save(spy);
28
29         // Verifico que el Repository ha consultado el nombre de la tarea
30         verify(spy).getNombre();
31     }
32
33 }
34
```

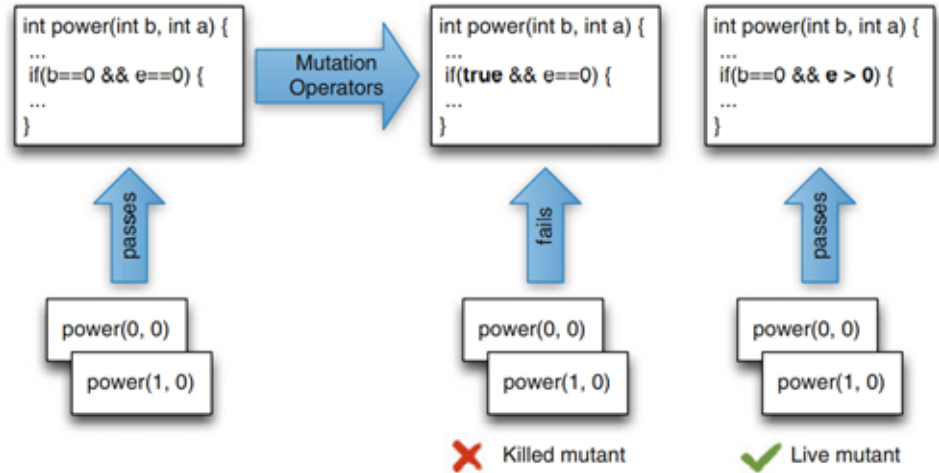
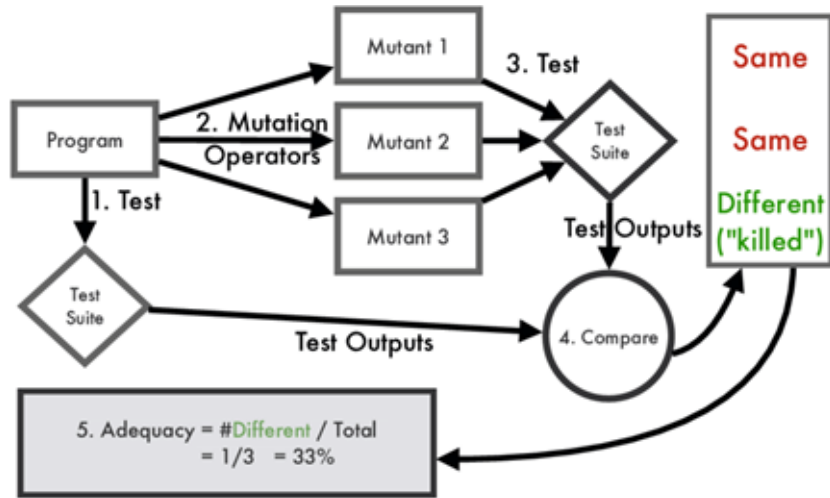


Fundamentos de pruebas de mutación

Las pruebas de mutación son una técnica de prueba de software que se utiliza para evaluar la calidad y efectividad de un conjunto de pruebas automatizadas.

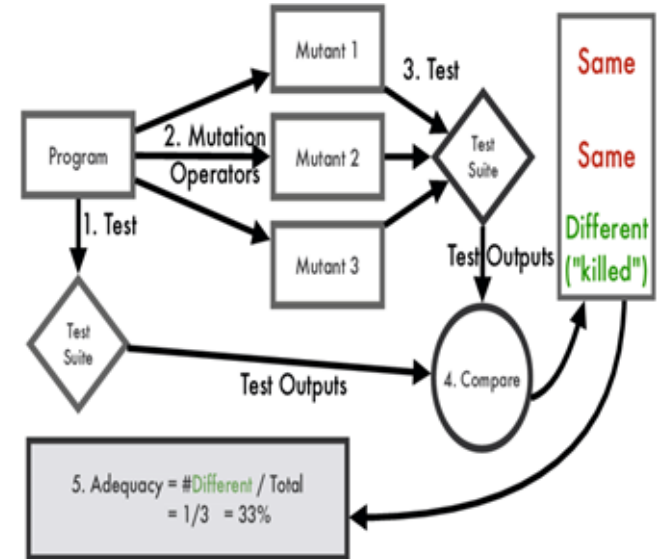


Mutation Testing



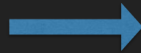
Mutation Testing

- Un **mutante** es una copia exacta de un programa o sistema, pero en el que introducimos un pequeño **fallo**.
- El objetivo de la mutación consiste en escribir casos de prueba que encuentren esos fallos que se siembran, insertan o inyectan en los mutantes
- Si el fallo se encuentra, el mutante está **muerto**; en otro caso, el mutante está **vivo**



Mutation Testing

```
if (a && b) {  
    c = 1;  
} else {  
    c = 0;  
}
```



```
if (a || b) {  
    c = 1;  
} else {  
    c = 0;  
}
```

El operador booleano && (and) es reemplazado por || (or). Este es un error común en todo tipo de aplicación.

Operadores de Mutación

ABS (absolute value)	Sustitución de una variable por el valor absoluto de dicha variable
ACR (array reference for constant replacement)	Sustitución de una referencia variable a un array por una constante
AOR (arithmetic operator replacement)	Sustitución de un operador aritmético
CRP (constant replacement)	Sustitución del valor de una constante
ROR (relational operator replacement)	Sustitución de un operador relacional
RSR (return statement replacement)	Sustitución de la instrucción Return
SDL (statement deletion)	Eliminación de una sentencia
UOI (unary operator insertion)	Inserción de operador unario (p.ej.: en lugar de x, poner -x)

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) && (a%100!= 0))
        b = true;
    return b;
}
```

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) || (a%100!= 0))
        b = true;
    return b;
}
```

Proceso de pruebas mediante mutación

1. Partimos de P (programa bajo prueba) y T (el *test suite*).
2. Generamos el conjunto de mutantes: $M = \{m_1, m_2, \dots, m_n\}$.
3. Ejecutamos T contra todos los mutantes vivos que haya en M .
4. Eliminamos los mutantes equivalentes.
5. Calculamos el *mutation score*.
6. Sacamos de M el conjunto de mutantes muertos.
7. Eliminamos los casos de prueba inefectivos (los que no matan ningún mutante) y:
 - 7.1 Si se alcanzó en el punto 5 el *mutation score* deseado, entonces se ejecuta T contra P :
 - 7.1.1 Si T (que es completo, porque alcanza el *mutation score*) no encuentra errores en P , termina el proceso.
 - 7.1.2 Si T sí encuentra errores, entonces se corrige P y, dado que P ha cambiado, se vuelve al paso 2, repitiendo el proceso.
 - 7.2 Si no se llega al *mutation score* deseado, entonces hay que revisar los casos de prueba existentes (tal vez no tengan un oráculo suficientemente bien descrito como para detectar los cambios de estado) y quizás añadir nuevos casos de prueba a T y volver al paso 3

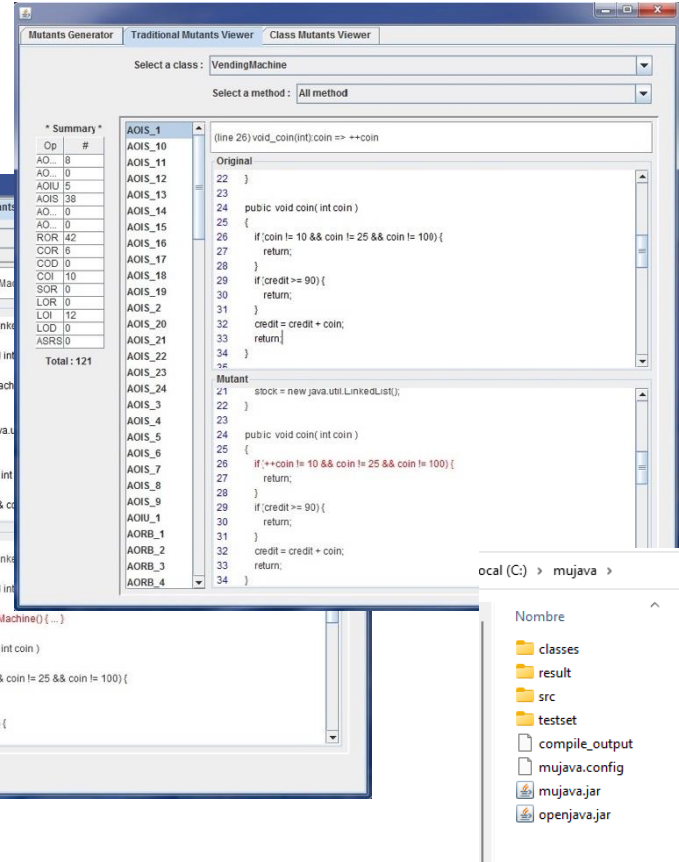
Ejemplo

Versión	Código
P (original)	<pre>int sum(int a, int b) { return a + b; }</pre>
Mutante 1	<pre>int sum(int a, int b) { return a - b; }</pre>
Mutante 2	<pre>int sum(int a, int b) { return a * b; }</pre>
Mutante 3	<pre>int sum(int a, int b) { return a / b; }</pre>
Mutante 4	<pre>int sum(int a, int b) { return a + b++; }</pre>

	Datos de prueba			
	(1, 1)	(0, 0)	(-1, 0)	(-1, -1)
P	2	0	-1	-2
Mutante 1	0	0	-1	0
Mutante 2	1	0	0	1
Mutante 3	1	Error	Error	1
Mutante 4	2	0	-1	-2

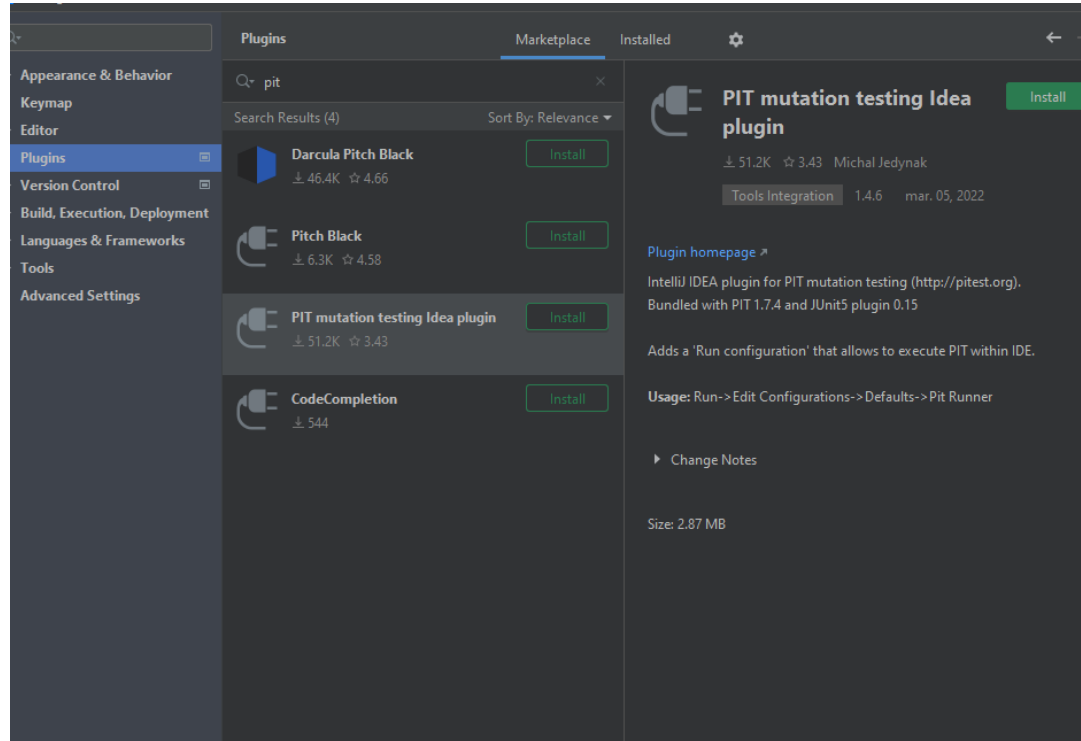
$$MS(P,T) = K / (M-E)$$

Cálculo del mutation score



```
set
CLASSPATH=%CLASSPATH%;C:\mujava\mujava.jar;C:\mujava\openjava.jar
;C:\Program Files\Java\jdk1.8.0_111\lib\tools.jar;C:\mujava\classes
```

Instalar PIT



The screenshot shows the IntelliJ IDEA Plugins Marketplace interface. The left sidebar contains a list of categories: Appearance & Behavior, Keymap, Editor, Plugins (selected), Version Control, Build, Execution, Deployment, Languages & Frameworks, Tools, and Advanced Settings. The main area is titled 'Plugins' and has tabs for 'Marketplace' and 'Installed'. A search bar at the top of the Marketplace tab contains the text 'pit'. Below the search bar, it says 'Search Results (4)' and 'Sort By: Relevance'. There are four search results listed, each with a plugin icon, name, download count, rating, and an 'Install' button:

- Darcula Pitch Black**: 46.4K downloads, 4.66 rating.
- Pitch Black**: 6.3K downloads, 4.58 rating.
- PIT mutation testing Idea plugin**: 51.2K downloads, 3.43 rating.
- CodeCompletion**: 544 downloads.

The right panel shows the details for the 'PIT mutation testing Idea plugin'. It includes the plugin icon, name, download count (51.2K), rating (3.43), author (Michal Jedynak), version (1.4.6), and release date (mar. 05, 2022). There is an 'Install' button. Below this, there is a link to the 'Plugin homepage', a description: 'IntelliJ IDEA plugin for PIT mutation testing (http://pitest.org). Bundled with PIT 1.7.4 and JUnit5 plugin 0.15', and a note: 'Adds a 'Run configuration' that allows to execute PIT within IDE.' There is also a 'Usage' section: 'Run->Edit Configurations->Defaults->Pit Runner' and a 'Change Notes' link. The size of the plugin is listed as 2.87 MB.

Calculadora.java

```
public class Calculadora {  
    public static int sumar(int num1, int num2) {  
        return num1 + num2;  
    }  
  
    public static int restar(int num1, int num2) {  
        return num1 - num2;  
    }  
  
    public static int multiplicar(int num1, int num2) {  
        return num1 * num2;  
    }  
  
    public static int dividir(int num1, int num2) {  
        return num1 / num2;  
    }  
}
```



Pruebas unitarias

```
public class CalculadoraTest {
    private Calculadora sut;

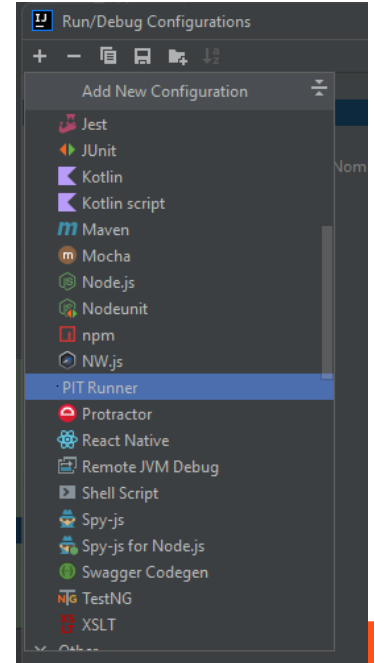
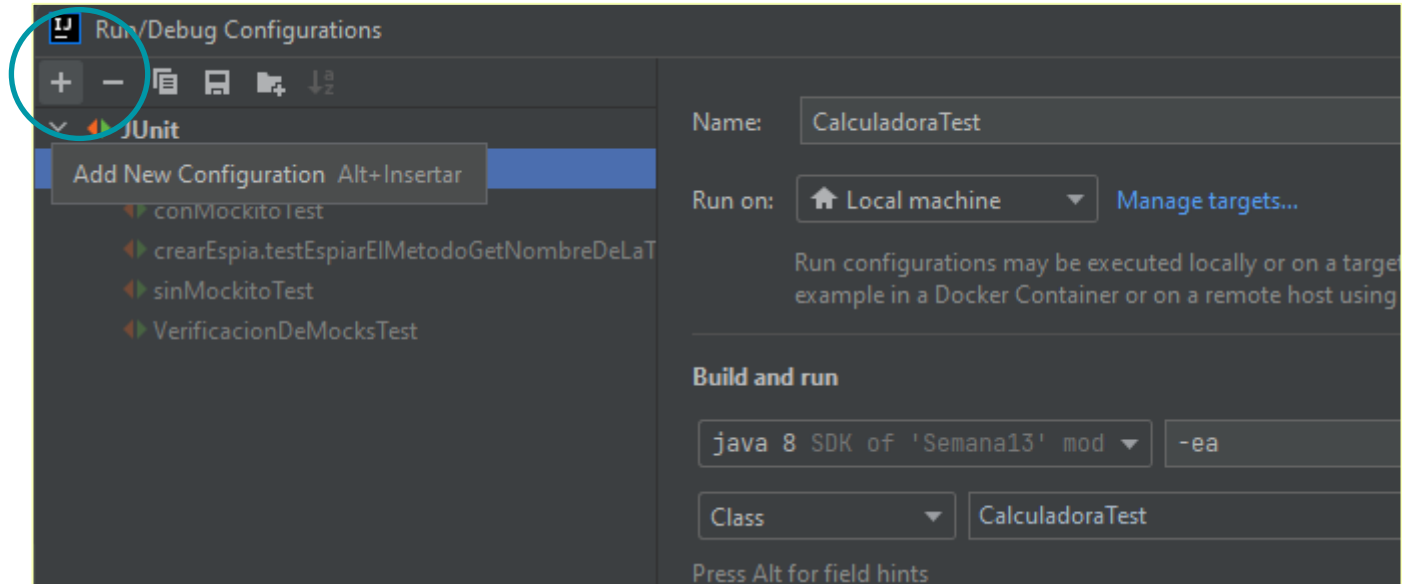
    @Before
    public void inicializa(){
        this.sut = new Calculadora();
    }

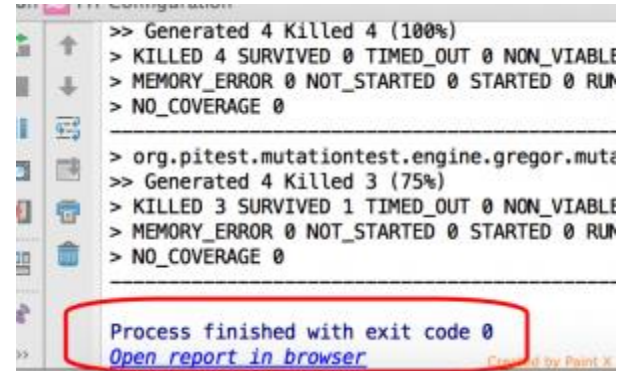
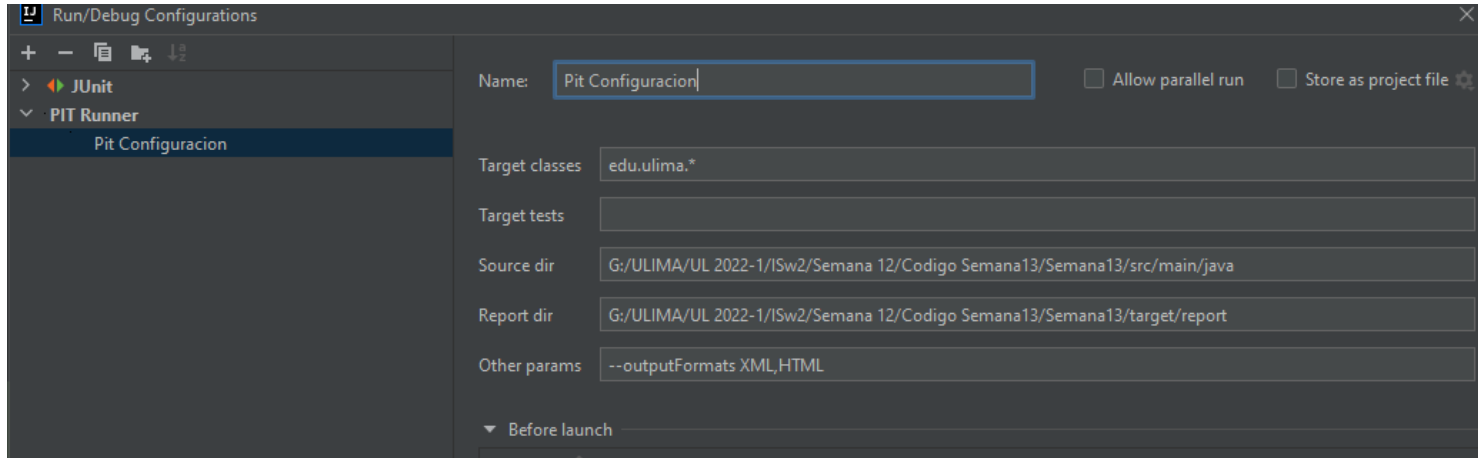
    @Test
    public void suma_debe_devolver_la_suma_de_los_operandos(){
        assertThat(4,is(this.sut.sumar(2, 2)));
    }

    @Test
    public void resta_debe_devolver_la_resta_de_los_operandos(){
        assertThat(6,is(this.sut.restar(8, 2)));
    }

    @Test
    public void multiplica_debe_devolver_la_multiplicacion_de_los_operandos(){
        assertThat(1,is(this.sut.multiplicar(1, 1)));
    }

    @Test
    public void divide_debe_devolver_la_division_de_los_operandos(){
        assertThat(5,is(this.sut.dividir(40, 8)));
    }
}
```





Pit Test Coverage Report

Package Summary

com.ejemplos

Number of Classes	Line Coverage	Mutation Coverage
1	100% 5/5	88% 7/8

Breakdown by Class

Name	Line Coverage	Mutation Coverage
Calculadora.java	100% 5/5	88% 7/8

Report generated by [PIT](#) 1.1.4

```

2
3 public class Calculadora
4 {
5     public int suma(int operando1, int operando2){
6         return operando1 * operando2;
7     }
8
9     public int resta(int operando1, int operando2){
10        return operando1 - operando2;
11    }
12
13    public int multiplica(int operando1, int operando2){
14        return operando1 * operando2;
15    }
16
17    public int divide(int operando1, int operando2){
18        return operando1 / operando2;
19    }
20 }

```

Mutations

- 6
 - 1. Replaced integer multiplication with division → KILLED
 - 2. replaced return of integer sized value with (x == 0 ? 1 : 0) → KILLED
- 10
 - 1. Replaced integer subtraction with addition → KILLED
 - 2. replaced return of integer sized value with (x == 0 ? 1 : 0) → KILLED
- 14
 - 1. Replaced integer multiplication with division → SURVIVED
 - 2. replaced return of integer sized value with (x == 0 ? 1 : 0) → KILLED
- 18
 - 1. Replaced integer division with multiplication → KILLED
 - 2. replaced return of integer sized value with (x == 0 ? 1 : 0) → KILLED

Tipos de pruebas de integración

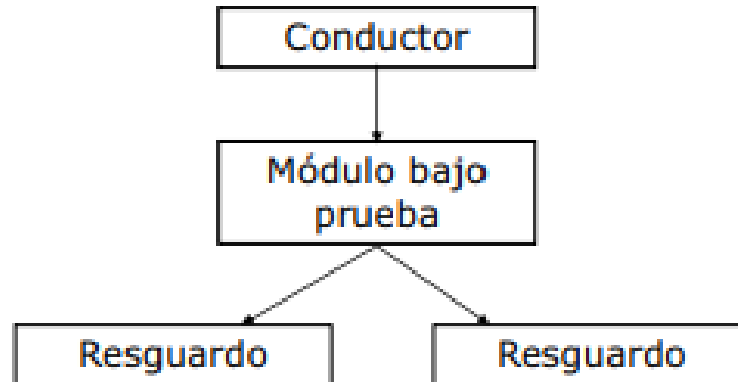
Las pruebas de integración son una fase crucial en el ciclo de vida del desarrollo de software, donde se combinan y prueban diferentes módulos o componentes del sistema para verificar su interacción y funcionamiento conjunto.

Tipos de pruebas

- **Prueba de Integración**
 - Las pruebas de integración son parte de las pruebas de sistema. Para integrar, se pueden usar diferentes técnicas:
 - TopDown
 - BottomUp
 - Sandwich
 - BigBang

TIPOS DE PRUEBAS

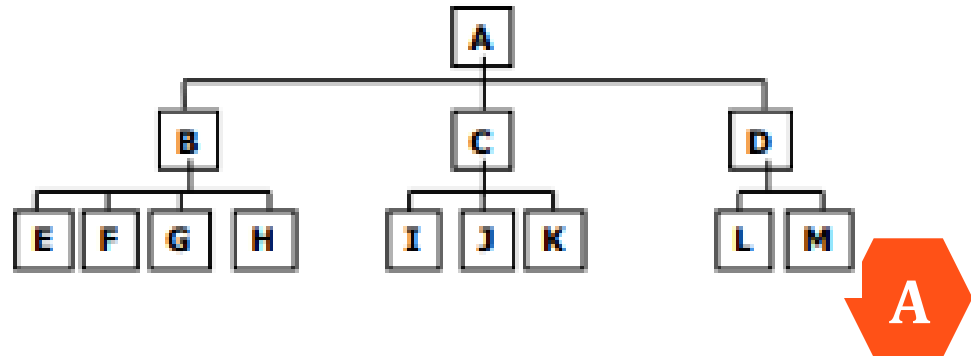
- Prueba de Integración
 - Pueden ser necesario escribir objetos simulados:
 - **Conductor (Test Driver)**: simulan la llamada al módulo pasándole los parámetros necesarios para realizar la prueba
 - **Resguardos (Stub)**: módulos simulados llamados por el módulo bajo prueba



Pruebas de Integración Top-Down

- Se comienza probando los módulos de nivel superior y se van integrando hacia abajo. Este método permite verificar la funcionalidad general del sistema desde el principio, utilizando stubs para simular los módulos de nivel inferior que aún no están disponibles.
 - Requiere muchos stubs de prueba.
 - Mayor abstracción

1. Probar el módulo A primero
2. Escribir módulos simulados, resguardos (stubs) para B, C, D
3. Una vez eliminados los problemas con A, probar el módulo B
4. Escribir resguardos para E, F, G, H etc.

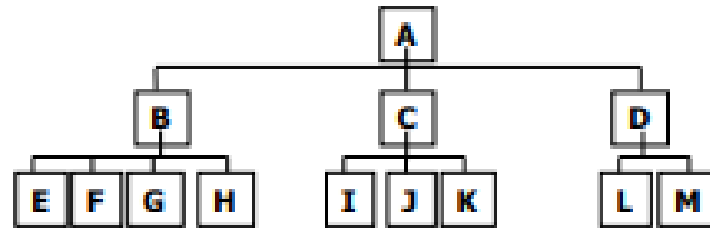


Pruebas de Integración Bottom-Up

Comienza con la integración y prueba de los módulos de nivel inferior, avanzando hacia los módulos de nivel superior. Se utilizan drivers para simular los módulos de nivel superior que aún no están disponibles.

- Errores que pueden posponerse hasta el final (puede ser costoso)
- Recomendable cuando hay reutilización.

1. Escribir conductores para proporcionar a E los datos que necesita de B
2. Probar E independientemente
3. Probar F independientemente, etc.
4. Una vez E, F, G, H han sido probados, el subsistema B puede ser probado, escribiendo un conductor para A

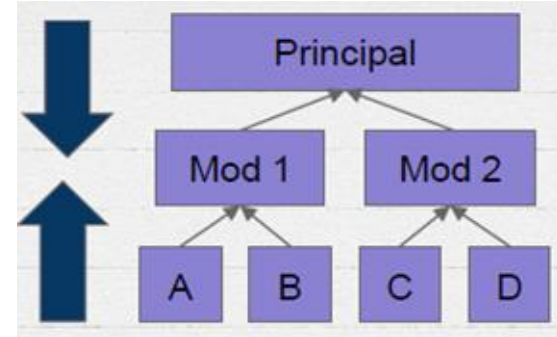


Pruebas de Integración Sandwich (o Híbridas)

El enfoque Sandwich combina los métodos Top-Down y Bottom-Up. Se integran y prueban simultáneamente los módulos de nivel superior e inferior, lo que permite una evaluación más equilibrada del sistema.

- Proporciona una visión completa del sistema desde el principio y permite detectar errores en diferentes niveles de manera simultánea.
- Puede ser más complejo de gestionar debido a la necesidad de coordinar ambos enfoques.

- El sistema se ve en 3 capas: Inferior, Media, Superior.
- Pruebas convergen en la capa media, donde están las funciones principales.

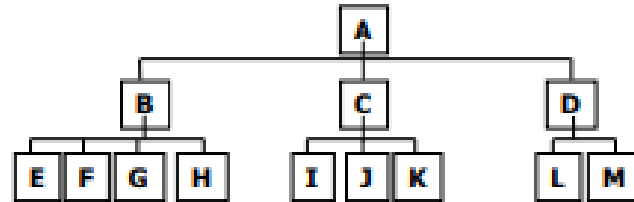


Pruebas de Integración Big Bang

Todos los módulos se integran y se prueban al mismo tiempo, después de que todos han sido desarrollados. Este método implica realizar una prueba exhaustiva del sistema completo.

- Puede ser más rápido en términos de configuración inicial, ya que no requiere pruebas incrementales.
- Dificulta la identificación de errores, ya que no se prueba la interacción de los módulos de manera incremental, lo que puede llevar a un proceso de depuración más complicado.

- Escribir y probar A, B, C, D, E, F, G, H, I, J, K, M a la vez.



Ejercicio de Estudio



Ejercicio 1 :

Implementa las pruebas unitarias para una clase de servicio que gestiona usuarios utilizando el framework Mockito. Deberás crear una interfaz llamada **UserRepository** con un método **User findUserById(int id)**, una clase **User** que represente a un usuario con atributos **id** y **name**, y una clase **UserService** que contenga un método **String getUsername(int id)**. Este método debe devolver el nombre del usuario correspondiente al ID proporcionado o "User not found" si el usuario no existe.

Crear una clase de prueba llamada **UserServiceTest** que utilizará JUnit y Mockito. En esta clase, configurarás un mock de **userRepository** y probarás el método **getUsername** en dos escenarios: cuando el usuario existe y cuando no. El objetivo de este ejercicio es familiarizarte con el uso de Mockito para simular el comportamiento de dependencias y verificar la lógica de tu código sin depender de implementaciones reales.



Solución :

```
1 // Interfaz del repositorio
2 public interface UserRepository {
3     User findUserById(int id);
4 }
5
6 // Clase de usuario
7 public class User {
8     private int id;
9     private String name;
10
11     public User(int id, String name) {
12         this.id = id;
13         this.name = name;
14     }
15
16     // Getters
17     public int getId() {
18         return id;
19     }
20
21     public String getName() {
22         return name;
23     }
24 }
25
26 // Clase del servicio
27 public class UserService {
28     private UserRepository userRepository;
29
30     public UserService(UserRepository userRepository) {
31         this.userRepository = userRepository;
32     }
33
34     public String getUserName(int id) {
35         User user = userRepository.findUserById(id);
36         return user != null ? user.getName() : "User not found";
37     }
38 }
```

Solución :

- **Configuración del Mock:** En el método `setUp`, creamos un mock de `UserRepository` utilizando `mock(UserRepository.class)`. Luego, inicializamos `UserService` con este mock.
- **Prueba de Usuario Existente:** En `testGetUserName_UserExists`, configuramos el mock para que devuelva un objeto `User` cuando se llame a `findUserId(1)`. Luego, verificamos que el nombre del usuario devuelto sea el esperado.
- **Prueba de Usuario No Encontrado:** En `testGetUserName_UserNotFound`, configuramos el mock para que devuelva `null` cuando se llame a `findUserId(2)`. Verificamos que el resultado sea "User not found".

```
1 import static org.junit.jupiter.api.Assertions.*;
2 import static org.mockito.Mockito.*;
3
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6
7 public class UserServiceTest {
8     private UserRepository userRepositoryMock;
9     private UserService userService;
10
11     @BeforeEach
12     public void setUp() {
13         userRepositoryMock = mock(UserRepository.class);
14         userService = new UserService(userRepositoryMock);
15     }
16
17     @Test
18     public void testGetUserName_UserExists() {
19         // Configurar el comportamiento del mock
20         User mockUser = new User(1, "John Doe");
21         when(userRepositoryMock.findById(1)).thenReturn(mockUser);
22
23         // Ejecutar el método a probar
24         String userName = userService.getUserName(1);
25
26         // Verificar el resultado
27         assertEquals("John Doe", userName);
28     }
29
30     @Test
31     public void testGetUserName_UserNotFound() {
32         // Configurar el comportamiento del mock
33         when(userRepositoryMock.findById(2)).thenReturn(null);
34
35         // Ejecutar el método a probar
36         String userName = userService.getUserName(2);
37
38         // Verificar el resultado
39         assertEquals("User not found", userName);
40     }
41 }
```

Ejercicio 2 :

En este ejercicio, analizarás un código existente que implementa un sistema de gestión de pedidos. Deberás identificar las clases que requieren mocks y crear pruebas unitarias utilizando Mocks.

crear una clase de prueba llamada OrderTest que utilice JUnit y Mockito. En esta clase, implementarás al menos dos pruebas unitarias para el método calculateTotalPrice de la clase Order:

- **Prueba de Producto Existente:** Configura un mock de ProductRepository para que devuelva un objeto Product válido cuando se llame a findProductById con un ID específico. Verifica que el método calculateTotalPrice devuelva el precio total correcto.
- **Prueba de Producto No Encontrado:** Configura el mock para que devuelva null cuando se llame a findProductById con un ID que no existe. Verifica que el método calculateTotalPrice devuelva 0.0.



```
1 // Clase que representa un producto
2 public class Product {
3     private int id;
4     private String name;
5     private double price;
6
7     public Product(int id, String name, double price) {
8         this.id = id;
9         this.name = name;
10        this.price = price;
11    }
12
13    public double getPrice() {
14        return price;
15    }
16 }
17
18 // Interfaz del repositorio de productos
19 public interface ProductRepository {
20     Product findProductById(int id);
21 }
22
23 // Clase que representa un pedido
24 public class Order {
25     private ProductRepository productRepository;
26
27     public Order(ProductRepository productRepository) {
28         this.productRepository = productRepository;
29     }
30
31     public double calculateTotalPrice(int productId, int quantity) {
32         Product product = productRepository.findProductById(productId);
33         if (product != null) {
34             return product.getPrice() * quantity;
35         }
36         return 0.0;
37     }
38 }
39
```

Solución :

```
1 import static org.junit.jupiter.api.Assertions.*;
2 import static org.mockito.Mockito.*;
3
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6
7 public class OrderTest {
8     private ProductRepository productRepositoryMock;
9     private Order order;
10
11     @BeforeEach
12     public void setUp() {
13         // Crear un mock de ProductRepository
14         productRepositoryMock = mock(ProductRepository.class);
15         // Inicializar Order con el mock
16         order = new Order(productRepositoryMock);
17     }
18
19     @Test
20     public void testCalculateTotalPrice_ProductExists() {
21         // Configurar el comportamiento del mock
22         Product mockProduct = new Product(1, "Test Product", 10.0);
23         when(productRepositoryMock.findProductById(1)).thenReturn(mockProduct);
24
25         // Ejecutar el método a probar
26         double totalPrice = order.calculateTotalPrice(1, 3); // 3 unidades
27
28         // Verificar el resultado
29         assertEquals(30.0, totalPrice);
30     }
31
32     @Test
33     public void testCalculateTotalPrice_ProductNotFound() {
34         // Configurar el comportamiento del mock
35         when(productRepositoryMock.findProductById(2)).thenReturn(null);
36
37         // Ejecutar el método a probar
38         double totalPrice = order.calculateTotalPrice(2, 5); // 5 unidades
39
40         // Verificar el resultado
41         assertEquals(0.0, totalPrice);
42     }
43 }
```

•**Configuración del Mock:** En el método setUp, se crea un mock de ProductRepository utilizando mock(ProductRepository.class). Luego, se inicializa la clase Order con este mock.

•**Prueba de Producto Existente:** En el método testCalculateTotalPrice_ProductExists, se configura el mock para que devuelva un objeto Product con un precio de 10.0 cuando se llama a findProductById(1). Luego, se llama al método calculateTotalPrice con un ID de producto y una cantidad de 3, y se verifica que el precio total devuelto sea 30.0.

•**Prueba de Producto No Encontrado:** En el método testCalculateTotalPrice_ProductNotFound, se configura el mock para que devuelva null cuando se llama a findProductById(2). Se llama al método calculateTotalPrice con un ID que no existe y se verifica que el resultado sea 0.0.

Ejercicio 3 :

Implemente las Pruebas de Mutación en un Sistema de Cálculo de Descuentos, en este ejercicio, deberás implementar una clase que calcule descuentos en productos y realizar pruebas de mutación para evaluar la efectividad de tus pruebas unitarias.

Parte 1: Implementación de la Clase

- Crea una clase llamada DiscountCalculator que contenga el siguiente método:
 - `public double calculateDiscount(double originalPrice, double discountPercentage)`: Este método debe recibir el precio original de un producto y un porcentaje de descuento, y devolver el precio después de aplicar el descuento. Debe lanzar una excepción `IllegalArgumentException` si el precio original es negativo, si el porcentaje de descuento es negativo o si el porcentaje de descuento es mayor a 100.
 - La fórmula para calcular el precio después del descuento es:

$$\text{precio final} = \text{precio original} - \left(\text{precio original} \times \left(\frac{\text{porcentaje de descuento}}{100} \right) \right)$$



Ejercicio 3 :

Parte 2: Pruebas Unitarias

- Implementa una clase de pruebas unitarias llamada DiscountCalculatorTest que contenga pruebas para el método calculateDiscount. Asegúrate de cubrir los siguientes casos:
 - Un precio válido y un porcentaje de descuento válido.
 - Un precio válido y un porcentaje de descuento de 0.
 - Un precio válido y un porcentaje de descuento de 100.
 - Un precio negativo, que debe lanzar una excepción.
 - Un porcentaje de descuento negativo, que debe lanzar una excepción.
 - Un porcentaje de descuento mayor a 100, que debe lanzar una excepción.



Ejercicio 3 :

Parte 3: Pruebas de Mutación

- Genera al menos cinco mutantes de la clase DiscountCalculator que introduzcan cambios en la lógica del método calculateDiscount. Los mutantes pueden incluir:
 - Cambios en los operadores de comparación en las validaciones.
 - Modificaciones en la fórmula de cálculo del descuento.
 - Cambios en los mensajes de excepción lanzados.
- Ejecuta las pruebas unitarias contra los mutantes generados y verifica si las pruebas son capaces de detectar los cambios introducidos. Documenta los resultados de las pruebas, indicando cuáles mutantes fueron detectados y cuáles no.

Solución :

Código de la Clase **DiscountCalculator**

```
1 public class DiscountCalculator {  
2     public double calculateDiscount(double originalPrice, double discountPercentage) {  
3         if (originalPrice < 0 || discountPercentage < 0 || discountPercentage > 100) {  
4             throw new IllegalArgumentException("Invalid price or discount percentage");  
5         }  
6         return originalPrice - (originalPrice * (discountPercentage / 100));  
7     }  
8 }
```



Solución :

Pruebas Unitarias: **DiscountCalculatorTest**

```
1  import static org.junit.jupiter.api.Assertions.*;
2  import org.junit.jupiter.api.Test;
3
4  public class DiscountCalculatorTest {
5      private DiscountCalculator discountCalculator = new DiscountCalculator();
6
7      @Test
8      public void testCalculateDiscount_ValidInput() {
9          double result = discountCalculator.calculateDiscount(100.0, 20.0);
10         assertEquals(80.0, result, 0.01);
11     }
12
13     @Test
14     public void testCalculateDiscount_ZeroDiscount() {
15         double result = discountCalculator.calculateDiscount(100.0, 0.0);
16         assertEquals(100.0, result, 0.01);
17     }
18
19     @Test
20     public void testCalculateDiscount_OneHundredPercentDiscount() {
21         double result = discountCalculator.calculateDiscount(100.0, 100.0);
22         assertEquals(0.0, result, 0.01);
23     }
24
25     @Test
26     public void testCalculateDiscount_NegativePrice() {
27         Exception exception = assertThrows(IllegalArgumentException.class, () -> {
28             discountCalculator.calculateDiscount(-100.0, 20.0);
29         });
30         assertEquals("Invalid price or discount percentage", exception.getMessage());
31     }
32
33     @Test
34     public void testCalculateDiscount_NegativeDiscount() {
35         Exception exception = assertThrows(IllegalArgumentException.class, () -> {
36             discountCalculator.calculateDiscount(100.0, -20.0);
37         });
38         assertEquals("Invalid price or discount percentage", exception.getMessage());
39     }
40
41     @Test
42     public void testCalculateDiscount_DiscountOverOneHundred() {
43         Exception exception = assertThrows(IllegalArgumentException.class, () -> {
44             discountCalculator.calculateDiscount(100.0, 120.0);
45         });
46         assertEquals("Invalid price or discount percentage", exception.getMessage());
47     }
48 }
```

Solución :

Mutante 1: Cambiar el operador de comparación para el precio negativo

```
public class DiscountCalculator {
    public double calculateDiscount(double originalPrice, double discountPercentage) {
        // Cambiar < a <=
        if (originalPrice <= 0 || discountPercentage < 0 || discountPercentage > 100) {
            throw new IllegalArgumentException("Invalid price or discount percentage");
        }
        return originalPrice - (originalPrice * (discountPercentage / 100));
    }
}
```

Mutante 2: Cambiar el operador de comparación para el porcentaje de descuento negativo

```
public class DiscountCalculator {
    public double calculateDiscount(double originalPrice, double discountPercentage) {
        // Cambiar > a >=
        if (originalPrice < 0 || discountPercentage < 0 || discountPercentage >= 100) {
            throw new IllegalArgumentException("Invalid price or discount percentage");
        }
        return originalPrice - (originalPrice * (discountPercentage / 100));
    }
}
```

Mutante 3: Cambiar el operador de comparación para el porcentaje de descuento mayor a 100

```
public class DiscountCalculator {
    public double calculateDiscount(double originalPrice, double discountPercentage) {
        // Cambiar < a <=
        if (originalPrice < 0 || discountPercentage <= 0 || discountPercentage > 100) {
            throw new IllegalArgumentException("Invalid price or discount percentage");
        }
        return originalPrice - (originalPrice * (discountPercentage / 100));
    }
}
```

Mutante 4: Modificar la fórmula de cálculo del descuento

```
public class DiscountCalculator {
    public double calculateDiscount(double originalPrice, double discountPercentage) {
        if (originalPrice < 0 || discountPercentage < 0 || discountPercentage > 100) {
            throw new IllegalArgumentException("Invalid price or discount percentage");
        }
        // Cambiar la fórmula de cálculo
        return originalPrice - (originalPrice * (discountPercentage / 200)); // Error
        introducido
    }
}
```

Mutante 5: Cambiar el retorno en caso de precio negativo

```
public class DiscountCalculator {
    public double calculateDiscount(double originalPrice, double discountPercentage) {
        if (originalPrice < 0 || discountPercentage < 0 || discountPercentage > 100) {
            // Cambiar el mensaje de excepción
            throw new IllegalArgumentException("Price cannot be negative"); // Mensaje
            cambiado
        }
        return originalPrice - (originalPrice * (discountPercentage / 100));
    }
}
```

¿Qué se logró?



- ✓ Comprender cómo utilizar mocks para simular comportamientos de objetos en pruebas unitarias, lo que les permitirá aislar componentes y verificar interacciones de manera efectiva.
- ✓ Aprender a implementar pruebas de mutación, evaluando la efectividad de sus pruebas automatizadas al introducir cambios en el código y verificando si las pruebas pueden detectarlos.
- ✓ Diferenciar entre los diversos tipos de pruebas de integración,, comprendiendo sus enfoques, ventajas y desventajas.

¿Qué debemos hacer para alcanzar el objetivo del tema?



- ✓ Repasar la teoría, los ejemplos y casos resueltos.



Referencias Bibliográficas

- Proceso de Construcción de Software 2, Mag. Natalí Flores Lafosse, Maestría en Informática de la PUCP, 2018.
- Sommerville, I. (2005). Ingeniería del software. Pearson educación.
- Bourque, P., & Fairley, R. E. (2014). Guide to the software engineering body of knowledge (SWEBOK (R)): Version 3.0. IEEE Computer Society Press.
- Doria, H. G. (2001). Las Metricas de Software y su Uso en la Region.

GRACIAS